

ARCANE: Adaptive Resilience for Cyber-Attacks in Node-level Embodiments

Anonymous Author(s)

Abstract

A clear and well-documented $\text{\LaTeX}$ document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the “acmart” document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.¹

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper;** *Generate the Correct Terms for Your Paper;* Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

ACM Reference Format:

Anonymous Author(s). 2018. ARCANE: Adaptive Resilience for Cyber-Attacks in Node-level Embodiments. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

2 Related Work

- Prior work on CPS security and resilience
- test [1]
- FDIR comparison/differences.

3 Resilience Framework

The criticality mapping is the backbone of the framework. Wrong estimations would affect the overall system resilience. The task-criticality mapping $\tau : D \times T \rightarrow C$ and the goal-criticality mapping $\varepsilon : D \times G \rightarrow C$ both have as codomain the set $C = \{0, 1, 2\}$, where 0 denotes no criticality, 1 denotes important, and 2 denotes required.

¹The full reproducible implementation is available: <https://github.com/mahyankashani/examensarbete>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-XXXX-X/2018/06 <https://doi.org/XXXXXXX.XXXXXXX>

These values are assigned to each device and are strictly related to either the task or the goal.

3.0.1 IDS. The IDS is a probabilistic function $I : D \rightarrow [0, 1]$ that assigns to each device an estimation of a possible ongoing cyberattack. Let $T_{\text{act}} \subseteq T$ and $G_{\text{act}} \subseteq G$ denote the currently active tasks and goals. A device d is considered critical if:

$$\exists t \in T_{\text{act}} : \tau(d, t) = 2 \vee \exists g \in G_{\text{act}} : \varepsilon(d, g) = 2 \quad (1)$$

The IDS confidence is compared against a device-specific threshold:

$$\kappa(d) = \begin{cases} \kappa_{\text{crit}}, & \text{if } d \text{ is critical} \\ \kappa_{\text{base}}, & \text{otherwise} \end{cases} \quad (2)$$

where $\kappa_{\text{crit}} < \kappa_{\text{base}}$. The stricter threshold κ_{crit} is applied to devices required for active tasks or goals, and κ_{base} in all other cases. A device is compromised and included in S when its IDS confidence meets or exceeds its device-specific threshold:

$$S = \{d \in D \mid I(d) \geq \kappa(d)\} \quad (3)$$

3.0.2 Tolerable Disruption. The tolerable disruption predicate $\delta : 2^D \rightarrow \{0, 1\}$ evaluates whether the compromised set S contains any device required for active tasks or goals:

$$\delta(S) = 1 \iff \forall t \in T_{\text{act}}, \nexists d \in S : \tau(d, t) = 2 \\ \wedge \forall g \in G_{\text{act}}, \nexists d \in S : \varepsilon(d, g) = 2 \quad (4)$$

That is, $\delta(S) = 1$ if and only if no compromised device is required by any active task or goal; otherwise $\delta(S) = 0$.

3.0.3 Tolerable Degradation. The tolerable degradation predicate $\gamma : 2^D \rightarrow \{0, 1\}$ is evaluated by comparing a monotonic non-increasing function $\psi : 2^D \rightarrow [0, 1]$ against a context-dependent threshold:

$$\gamma(S) = 1 \iff \psi(S) \geq \theta(S) \quad (5)$$

where:

$$\theta(S) = \begin{cases} \theta_{\text{crit}}, & \text{if } \exists d \in S : d \text{ is critical} \\ \theta_{\text{base}}, & \text{otherwise} \end{cases} \quad (6)$$

with $\theta_{\text{crit}} > \theta_{\text{base}}$. The stricter threshold θ_{crit} is applied whenever a required device is present in S .

3.0.4 Mitigation Feasibility. The mitigation feasibility predicate $\mu : 2^D \rightarrow \{0, 1\}$ formalizes whether resilience can be restored through corrective actions:

$$\mu(S) = 1 \iff \exists M \subseteq S : \delta(S \setminus M) = 1 \wedge \gamma(S \setminus M) = 1 \quad (7)$$

That is, $\mu(S) = 1$ if and only if there exists a subset $M \subseteq S$ that can be neutralized such that the remaining set $S \setminus M$ satisfies both disruption and degradation tolerance simultaneously.

3.0.5 System Resilience. The system resilience guarantee states that, given a compromised set S , the system is resilient if and only if:

$$\begin{aligned} &(\delta(S) = 1 \wedge \gamma(S) = 1) \\ &\vee (\mu(S) = 1 \wedge \delta(S \setminus M) = 1 \wedge \gamma(S \setminus M) = 1) \end{aligned} \quad (8)$$

Runtime resilience evaluation produces three distinct cases. In the base case, $\delta(S) = 1 \wedge \gamma(S) = 1$: the system is resilient and no mitigation is required. When $\mu(S) = 1$, the system restores resilience by neutralizing $M \subseteq S$ such that $\delta(S \setminus M) = 1 \wedge \gamma(S \setminus M) = 1$. In the third case, $\mu(S) = 0$ with $\delta(S) = 0 \vee \gamma(S) = 0$: either disruption, degradation, or both are intolerable and no feasible mitigation exists, leaving the system in a non-resilient state.

4 Threat Model

We consider an embodied CPS executing a task $t \in T$ autonomously, relying only on its navigation, manipulation, and perception subsystems to complete the task. No human operator is involved. In addition to these operational capabilities, the system is equipped with the resilience framework described in Section 3, continuously evaluating δ , γ , and μ during execution.

4.1 Attacker Goal

The adversary seeks to force $\delta(S) = 0$ or $\gamma(S) = 0$ by compromising one or more devices $d \in D$, violating the resilience guarantee of Equation (8). Disruption may take the form of impeding navigation, interfering with object acquisition, or corrupting the robot's perception of the surrounding environment. The ultimate objective is to cause task failure, safety-relevant hazards such as collisions with nearby obstacles, or exhaustion of the available time budget.

4.2 Attacker Capabilities

The attacker operates at the ROS2 communication layer, injecting symbolic message-based attacks targeting one or more devices $d \in D$ simultaneously. For each targeted device, it selects an attack type based on its overall objective. For example, if its objective is to tamper with obstacle avoidance, it will target all devices involved in that subsystem, either individually or in a specific combination, to deviate from the original execution. Once an attack is active, it remains in force until a mitigation $M \subseteq S$ is deployed to resolve it, as formalized in Equation (7).

4.3 Attacker Knowledge

The attacker is modeled as an outsider with partial system knowledge. It knows which devices $d \in D$ exist and which attack types are applicable to each. However, it has no access to the criticality mappings τ and ε , the internals of the Resilience Manager, or the IDS function I defined in Section 3.0.1. It cannot tamper with their execution or observe their outputs. The resilience framework and the IDS are assumed trusted and uncompromised, reflecting a deployment where both components run on isolated, hardened infrastructure.

4.4 Defenses

The IDS is probabilistic, providing a per-device confidence score $I(d) \in [0, 1]$ that estimates the probability of an ongoing compromise. A device d enters the compromised set S when $I(d) \geq \kappa(d)$, as defined in Equations (2) and (3). The mitigations, when applied, are deterministic and fully resolve the ongoing attack for each targeted device, removing it from S and restoring the evaluation of $\delta(S \setminus M)$ and $\gamma(S \setminus M)$.

5 Implementation

The implementation translates the formal resilience framework presented in Section 3 into an executable evaluation system for embodied CPSs. The system operates continuously during task execution, reasoning about the current resilience state of the robot under cyberattack conditions consistent with the threat model of Section 4.

5.1 System Architecture

The framework comprises five modules: a simulation environment, a coordination controller, a task module, an attack injection module, and a resilience evaluation module. Figure 1 illustrates the overall architecture and the data flow at runtime.

During initialization, the controller loads the experiment configuration, which specifies task type, goal activation, criticality mappings τ and ε , system thresholds κ_{crit} , κ_{base} , θ_{crit} , θ_{base} , reduction coefficients α_{crit} and α_{base} , baseline execution time, and the mitigatable device set. These parameters are distributed to the IDS and Resilience Manager before task execution begins.

During runtime, the attacker operates at the ROS2 communication layer as described in Section 4.2. Symbolic attack messages are published to the `active_attacks` topic and received by a subscriber node inside the Webots simulation, which forwards the target device and attack type to both the Attack Executor and the IDS. The Resilience Manager evaluates the current system state based on the IDS output and returns mitigation decisions to the Attack Executor. The final task result and runtime metrics are collected by the controller at the end of execution.

The system is implemented in Python, using Webots as the simulation environment, a PR2 robot platform, and ROS2 for attack communication. A key integration challenge arose from hardware contention: the original Webots PR2 controller continuously overwrites motor commands during task execution, silently negating attack effects. This was resolved by inserting attack-awareness checks into the low-level hardware control layer, granting the Attack Executor temporary ownership of targeted actuators when an attack is active.

5.2 Framework Modules

Three modules implement the core resilience predicates defined in Section 3.

5.2.1 IDS. The IDS evaluates each attacked device $d \in D$ independently against its device-specific threshold $\kappa(d)$ defined in Equation (2). Devices satisfying $\tau(d, t) = 2$ or $\varepsilon(d, g) = 2$ for any active task or goal are evaluated against the stricter threshold κ_{crit} ; all

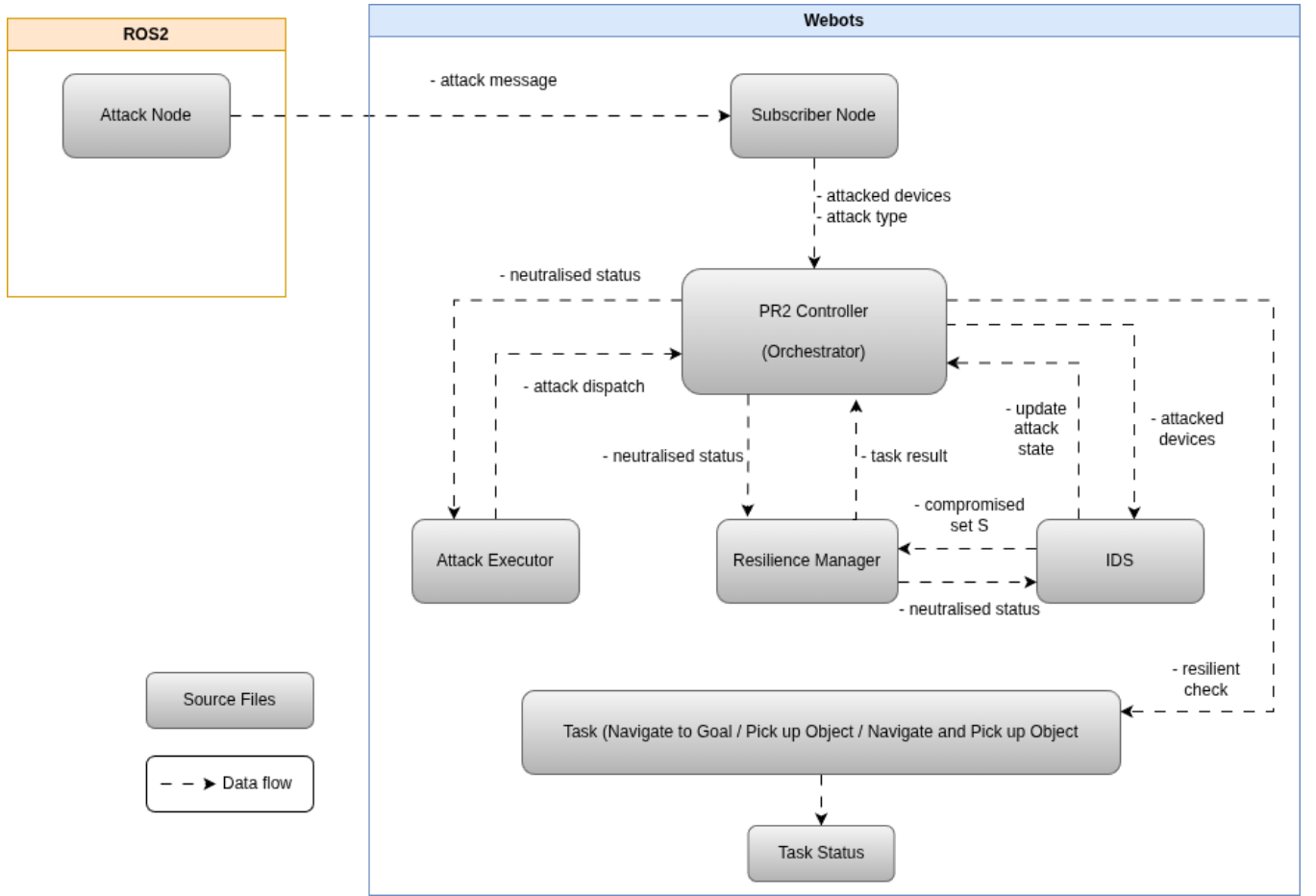


Figure 1: Overview of implementation and evaluation of the proposed framework

others use κ_{base} . To prevent repeated evaluation of the same ongoing attack from artificially inflating detection confidence over time, a tested-component set ensures each device is evaluated only once per attack event. When a device is neutralized by a mitigation action, it is cleared from S and re-enabled for future detection.

5.2.2 Resilience Manager. The Resilience Manager executes a continuous monitoring loop during task execution, receiving the current compromised set S from the IDS and evaluating δ , γ , and μ as defined in Equations (4), (5), and (7).

Four runtime states emerge from the combinations of δ and γ . In the base state, $\delta(S) = 1 \wedge \gamma(S) = 1$: the system is resilient and no mitigation is required, even with an ongoing attack. When $\delta(S) = 1 \wedge \gamma(S) = 0$, no required device is compromised but aggregate degradation exceeds θ_{base} ; mitigations are deployed to restore γ . When $\delta(S) = 0 \wedge \gamma(S) = 1$, a required device is compromised but $\psi(S)$ still exceeds θ_{crit} , representing an edge case where the threshold configuration tolerates the current disruption level; to maintain consistency with Equation (6), ψ should fall below θ_{crit} whenever at least one critical device is affected, and mitigations are required in any case to restore δ . When $\delta(S) = 0 \wedge \gamma(S) = 0$, both predicates are violated simultaneously, representing the most severe

state and corresponding to a significant or coordinated attack. In all states where $\delta(S) = 0 \vee \gamma(S) = 0$, the manager evaluates $\mu(S)$ as defined in Equation (7). When $\mu(S) = 1$, a mitigation timer introduces a non-instantaneous delay before the neutralized set $M \subseteq S$ is forwarded to the Attack Executor and removed from the active attack state. If no feasible mitigation exists, the task module returns HALTED.

5.2.3 Attack Executor. The Attack Executor manages attacks and their effects on devices $d \in D$, consistent with the attacker capabilities described in Section 4.2. Attacks are typed symbolically: STOP halts the targeted device entirely, UNDERSPEED reduces actuator velocity, FREEZE immobilizes arm joints, and WEAKGRIP reduces gripper force. During an active attack, the executor takes temporary hardware ownership of the targeted device, preventing normal task commands from overwriting attack effects. Devices present in the current neutralized set M are excluded from attack application.

5.3 ψ Function

The degradation function $\psi : 2^D \rightarrow [0, 1]$, introduced in Section 3.0.3, is required to be monotonic and non-increasing but is not prescribed by the framework. The instantiation choice determines

how aggressively compromised devices reduce the computed performance score, and therefore how quickly the system transitions into the degraded state $\gamma(S) = 0$. Let $k_{\text{crit}} = |\{d \in S \mid \tau(d, t) = 2\}|$ and $k_{\text{base}} = |\{d \in S \mid \tau(d, t) = 1\}|$ denote the number of compromised critical and non-critical devices respectively. Two candidate forms are evaluated. The linear form decreases proportionally with the number of compromised devices:

$$\psi_{\text{mono}}(S) = \max(0, 1 - \alpha_{\text{crit}} \cdot k_{\text{crit}} - \alpha_{\text{base}} \cdot k_{\text{base}}) \quad (9)$$

The exponential form applies a steeper early penalty for critical device compromise:

$$\psi_{\text{exp}}(S) = e^{-(\alpha_{\text{crit}} \cdot k_{\text{crit}} + \alpha_{\text{base}} \cdot k_{\text{base}})} \quad (10)$$

Because D is finite, both instantiations produce step functions over the discrete set of device subsets regardless of the underlying continuous form. The exponential form ψ_{exp} was selected for the experiments presented in Section 7, as its steeper initial slope better reflects the disproportionate operational impact of losing the first critical device. The specific parameter configurations used in the evaluation are reported in Section 6.

6 Methodology

The evaluation is designed to verify consistency between the implementation and the theoretical framework of Section 3, and to characterize how the configurable parameters introduced in Sections 3 and 5 influence runtime resilience behavior under the attack conditions defined in Section 4.

By designing the following experiments, we would like to answer three research questions.

RQ₁: How can we measure the performance of the proposed IDS under various cyber attacks?

RQ₂: What is the suitable detection rate of the proposed IDS?

RQ₃: Is an attacker capable of deliberately and efficiently attacking a CPS, which is developed in a physics-based simulation with ROS2 communication middleware?

6.1 Experimental Setup

All experiments are conducted in a Webots simulation environment using a PR2 robot platform. The simulation scene comprises the robot, two wooden boxes serving as navigation targets, one table containing a graspable object, and one empty table used as a placement target. No physical hardware is used; the evaluation is entirely simulation-based.

Each experiment is configured via a JSON file loaded by the controller at initialization, as described in Section 5.1. The configuration specifies task type, goal activation, baseline execution time, criticality mappings τ and ε , system thresholds κ_{crit} and κ_{base} , degradation thresholds θ_{crit} and θ_{base} , reduction coefficients α_{crit} and α_{base} , and the mitigatable device set. This configuration-driven design allows all parameters to be varied between experiments without modifying the framework code.

Attacks are manually triggered during task execution by publishing symbolic messages to the ROS2 `active_attacks` topic, consistent with the attacker capabilities of Section 4.2. Each attack message specifies one or more target devices $d \in D$ and their associated attack type. The framework logs all runtime state transitions, including updates to the compromised set S , evaluations of δ and

γ , mitigation events, and task outcomes. Results are stored in CSV files for post-hoc analysis. The collected metrics per experiment are: task result (DONE or HALTED), execution time, degradation percentage relative to baseline, and the compromised set S at each evaluation step.

6.2 Experiment Design

Four experiments are conducted, each targeting a distinct aspect of the framework behavior.

6.2.1 Framework Correctness. The purpose is to verify that the implementation correctly transitions between all combinations of δ and γ defined in Section 5.2.2. To isolate the resilience logic from IDS uncertainty, both κ_{crit} and κ_{base} are set to 0.0, ensuring that $I(d) \geq \kappa(d)$ holds for all attacked devices and the compromised set S (Equation (3)) is always fully populated. All scenarios use the `navigate_and_pickup` task, which involves both navigation and manipulation and therefore produces the largest critical device set among the available tasks. No embodiment-related goals are active. Four attack scenarios are evaluated:

- a non-critical device ($\tau(d, t) < 2$) attacked without available mitigation, expected to preserve $\delta(S) = 1$;
- a critical device ($\tau(d, t) = 2$) attacked without available mitigation, expected to produce $\delta(S) = 0 \wedge \gamma(S) = 0$;
- two critical devices attacked with available mitigation $M \subseteq S$, expected to restore $\delta(S \setminus M) = 1 \wedge \gamma(S \setminus M) = 1$ via Equation (7);
- an UNDERSPEED attack on two critical devices without mitigation, designed to produce performance degradation $\psi(S) < \theta_{\text{crit}}$ without necessarily causing task failure.

Table 1: Framework correctness attack scenarios. All experiments use the `navigate_to_goal` task. Unaffected devices are omitted; see Table 2 for the full device list.

S#	Target	$\tau(d, t)$	Attack	Mitigation
1	right_arm	1	STOP	No
2	left_arm	2	STOP	No
3	left_wheels	2	STOP	Yes
	right_wheels	2	STOP	Yes
4	left_wheels	2	UNDERSPEED	No
	right_wheels	2	UNDERSPEED	No

$\tau \in \{0, 1, 2\}$: 0 = none, 1 = important, 2 = required.

6.2.2 IDS Sensitivity. The purpose is to characterize how imperfect detection propagates into resilience misclassification. Two critical devices are attacked simultaneously during the `navigate_and_pickup` task. The detection rate is varied by setting κ_{crit} to 0.3, 0.2, and 0.1, corresponding to per-device detection probabilities of 70%, 80%, and 90% respectively, while κ_{base} is held fixed. For each configuration, 20 simulation runs are conducted. The compromised set S (Equation (3)) is logged at each run to record whether neither, one, or both attacked devices were detected, and how partial detection affects the evaluation of δ and γ .

6.2.3 Tolerable Degradation. The purpose is to map the parameter space of degradation tolerance under varying attack severity. Only critical devices ($\tau(d, t) = 2$) are attacked, forcing $\delta(S) = 0$ throughout and ensuring γ is always evaluated against θ_{crit} as defined in Equation (6). The reduction coefficient α_{crit} is swept across $\{0.1, 0.2, 0.3\}$ in three separate parameter sweeps, while θ_{crit} is varied from 0.0 to 1.0. The number of compromised critical devices k_{crit} ranges from 1 to 5, covering all disruption levels for the `navigate_and_pickup` task. The degradation score $\psi(S)$ is computed analytically from the exponential instantiation ψ_{exp} defined in Equation (10) and compared against each θ_{crit} value to determine whether $\gamma(S) = 1$.

6.2.4 Task Sensitivity. The purpose is to quantify how task-dependent criticality mappings influence disruption likelihood under an attacker that knows D but not τ or ϵ , consistent with the knowledge model of Section 4.3. Rather than repeated simulation runs, this experiment performs a combinatorial analysis by exhaustively enumerating all $2^7 - 1 = 127$ non-empty subsets of D . For each task configuration, every subset is evaluated to determine whether it contains at least one device with $\tau(d, t) = 2$ or $\epsilon(d, g) = 2$, which by Equation (4) causes $\delta(S) = 0$. The three task configurations are evaluated independently.

6.3 Task and Criticality Configuration

Table 2 presents the task-criticality mapping $\tau(d, t)$ used across all experiments. The robot comprises seven devices $d \in D$ covering navigation, manipulation, and perception subsystems.

In addition to τ , the goal-criticality mapping ϵ is active for the obstacle avoidance goal in navigation-related tasks. Under this goal, $\epsilon(\text{distance_sensor}, g) = 2$, making the `distance_sensor` critical independently of its task-level assignment via Equation (1). This increases the effective critical device count and the attack surface for tasks involving navigation. The Task Sensitivity experiment includes this goal-criticality contribution to reflect the full criticality configuration used during evaluation.

7 Evaluation

7.1 Attack Scenarios

For each scenario, the corresponding runtime output is presented to illustrate how the implemented framework evaluates resilience during execution. The results are analyzed with respect to the expected behavior defined in the theoretical framework to verify that the implementation correctly transitions between resilient, disrupted, degraded, and non-resilient states.

Listing 1: Attack on non-critical component without mitigation

```
[RM] System not disrupted →  $\delta = 1$ 
[RM] Degradation within tolerance →  $\gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: right_arm (STOP) - NON-CRITICAL
[RM] Compromised set S updated: {'right_arm'}
```

```
Task result: DONE
```

The results confirm the expected framework behavior. Although the `right_arm` component was compromised, the task remained operational because the component was not classified as critical for task completion. Consequently, the system remained resilient throughout execution.

Listing 2: Attack on non-critical components without mitigation, Not tolerable after second attack

```
[RM] System not disrupted →  $\delta = 1$ 
[RM] Degradation within tolerance →  $\gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: right_wheels (STOP) - NON-CRITICAL
[RM] Compromised set S updated: {'right_wheels'}
```

```
ATTACK: right_wheels (STOP) - NON-CRITICAL
ATTACK: right_arm (STOP) - NON-CRITICAL
[RM] Compromised set S updated: {'right_arm', 'right_wheels'}
```

```
[RM] System degraded beyond tolerance:  $\psi=0.7 < \theta=0.8 \rightarrow \gamma = 0$ 
[RM] Resilience state: NOT RESILIENT
[RM] No mitigation possible - system irreparably compromised
Task result: HALTED
```

Listing 3: Attack on non-critical components lead into not tolerable state after second attack with mitigation

```
[RM] System not disrupted →  $\delta = 1$ 
[RM] Degradation within tolerance →  $\gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: right_wheels (STOP) - NON-CRITICAL
[RM] Compromised set S updated: {'right_wheels'}
```

```
ATTACK: right_wheels (STOP) - NON-CRITICAL
ATTACK: right_arm (STOP) - NON-CRITICAL
[RM] Compromised set S updated: {'right_arm', 'right_wheels'}
```

```
[RM] System degraded beyond tolerance:  $\psi=0.7 < \theta=0.8 \rightarrow \gamma = 0$ 
[RM] Resilience state: NOT RESILIENT
[RM] Mitigation pending
[RM] Active mitigation neutralizing: {'right_arm'}
```

```
[RM] Compromised set S updated: {'right_wheels'}
```

```
[RM] Degradation within tolerance →  $\gamma = 1$ 
[RM] Resilience state: RESILIENT
```

Unlike Scenario 1, compromising a critical component immediately caused the framework to classify the system as disrupted. Since no mitigation was available, the system transitioned into a non-resilient state and task execution was halted.

Table 2: Task-criticality mapping $\tau(d, t)$ across task types. $\tau \in \{0, 1, 2\}$: 0 = none, 1 = important, 2 = required.

Component	Role	pickup_object	navigate_to_goal	navigate_and_pickup
left_wheels	Navigation	0	2	2
right_wheels	Navigation	0	2	2
left_arm	Manipulation	2	0	2
right_arm	Manipulation	1	0	1
left_gripper	Manipulation	2	0	2
right_gripper	Manipulation	1	0	1
distance_sensor	Perception	0	2	2
Critical devices D_c		2	3	5

Listing 4: Attack on critical component without mitigation

```
[RM] System not disrupted  $\rightarrow \delta = 1$ 
[RM] Degradation within tolerance  $\rightarrow \gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: left_wheels (STOP) - CRITICAL
[RM] Compromised set S updated: {'left_wheels'}
[RM] System disrupted  $\rightarrow \delta = 0$ 
[RM] System degraded beyond tolerance:  $\psi=0.8 <$ 
 $\theta=0.9 \rightarrow \gamma = 0$ 
[RM] Resilience state: NOT RESILIENT
[RM] No mitigation possible - system
irreparably compromised
Task result: HALTED
```

Listing 5: Attack on critical component with mitigation

```
[RM] System not disrupted  $\rightarrow \delta = 1$ 
[RM] Degradation within tolerance  $\rightarrow \gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: left_wheels (STOP) - CRITICAL
[RM] Compromised set S updated: {'left_wheels'}
[RM] System disrupted  $\rightarrow \delta = 0$ 
[RM] System degraded beyond tolerance:  $\psi=0.8 <$ 
 $\theta=0.9 \rightarrow \gamma = 0$ 
[RM] Resilience state: NOT RESILIENT
[RM] Mitigation pending
[RM] Active mitigation neutralizing: {'left_wheels'}
[RM] Compromised set S updated: set()
[RM] System not disrupted  $\rightarrow \delta = 1$ 
[RM] Degradation within tolerance  $\rightarrow \gamma = 1$ 
[RM] Resilience state: RESILIENT
Task result: DONE
```

This experiment demonstrates that the framework correctly models the effect of mitigation during runtime. After the attack influence was removed from the compromised component, the system returned to a resilient state.

Listing 6: UNDERSPEED attack causing increased task execution

```
[RM] System not disrupted  $\rightarrow \delta = 1$ 
[RM] Degradation within tolerance  $\rightarrow \gamma = 1$ 
[RM] Resilience state: RESILIENT
ATTACK: left_wheels (UNDERSPEED) - CRITICAL
ATTACK: right_wheels (UNDERSPEED) - CRITICAL
[RM] Compromised set S updated: {'left_wheels',
'right_wheels'}
[RM] System disrupted  $\rightarrow \delta = 0$ 
[RM] System degraded beyond tolerance:  $\psi=0.6 <$ 
 $\theta=0.9 \rightarrow \gamma = 0$ 
[RM] Resilience state: NOT RESILIENT
[RM] No mitigation possible - system
irreparably compromised
Task result: DONE
Task execution time increased by 6.5 seconds
Degradation: 10.7%
```

The results show that disruption does not necessarily imply task failure. Despite degraded performance and increased execution time, the robot successfully completed the assigned task.

Overall, the results indicate that the implemented framework follows the expected resilience behavior defined in the theoretical framework. The evaluated attack scenarios demonstrate that the system correctly distinguishes between attacks targeting critical and non-critical components and transitions accordingly between resilient and non-resilient states.

Attacks on non-critical components did not disrupt task execution, while attacks targeting critical components resulted in disruption and, when mitigation was unavailable, non-resilient behavior and task termination.

When mitigation was available, the framework successfully modeled the effect of mitigation by removing attack influence from compromised components, allowing the system to return to a resilient state.

Finally, the UNDERSPEED scenario demonstrated that disruption does not necessarily imply task failure. Although performance degradation occurred and execution time increased, task completion

remained possible, illustrating the distinction between degraded and failed system behavior.

7.2 IDS Sensitivity

Figure 2 presents the results of the IDS sensitivity experiments for detection rates of 70%, 80% and 90%, corresponding to κ_{crit} values of 0.3, 0.2 and 0.1 respectively. Across all experiments, increasing the IDS detection rate improved the likelihood that compromised components were correctly identified during runtime.

For the 70% detection-rate configuration, IDS failed to detect both compromised components in 10% of simulation runs, detected only one compromised component in 50% of the runs, and successfully identified both compromised components in 40% of the runs. Increasing the detection rate to 80% produced similar results regarding completely undetected attacks, which still occurred in 10% of the runs, but the proportion of fully detected attacks increased to 55%.

The highest detection performance was observed for the 90% configuration, where the IDS detected at least one compromised component in all simulation runs. In this setup, both compromised components were detected simultaneously in 85% of the conducted experiments.

To understand how many experiments must be run, we tested using 11.

$$n = \left(\frac{z \cdot \sigma}{E} \right)^2 \quad (11)$$

where n is the number of experiments, σ is the standard deviation of the values psi , E is the error margin, and z is the z-score corresponding to the desired confidence level (0.90%, 0.95%, 0.99%), shown in Table 3.

Table 3: Z-scores for common confidence intervals

Confidence Level	Z-Score
90%	1.645
95%	1.960
99%	2.576

We investigate a different number of runs in one experiment to navigate the goal, *Box2* with these parameters setting: θ_{crit} , θ_{base} , α_{base} , α_{crit} , κ_{crit} , κ_{base} are 0.8, 0.5, 0.04, 0.3, 0.6 respectively.

According to experiments, for 3 common Confidence Intervals (CI), considering the different detection rates, there is enough sample size presented in Table 4.

According to our experiments for two 70% and 80% Detection Rate, 20 runs are sufficient for 95% CI with 0.05 Margin Error.

7.3 Tolerable Disruption and Degradation

The weighted exponential Monotonic degradation function is presented in 3. Timeline of the monotonic degradation function (ψ) for Experiment 9 presented in Fig 10. The plot illustrates the system's performance degradation over time, comparing mitigation versus

non-mitigation states under various attacks: left and right wheels, left and right arms, left gripper.

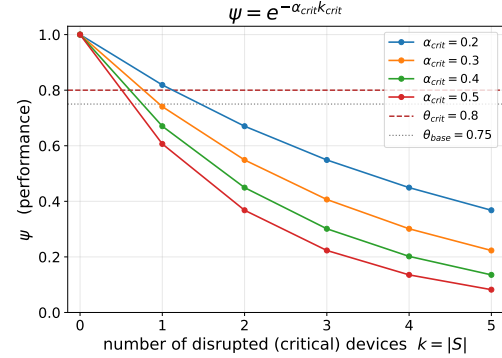


Figure 3: Monotonic degradation using weighted exponential function

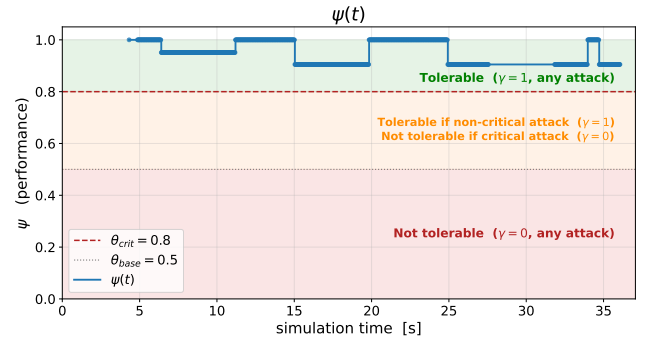


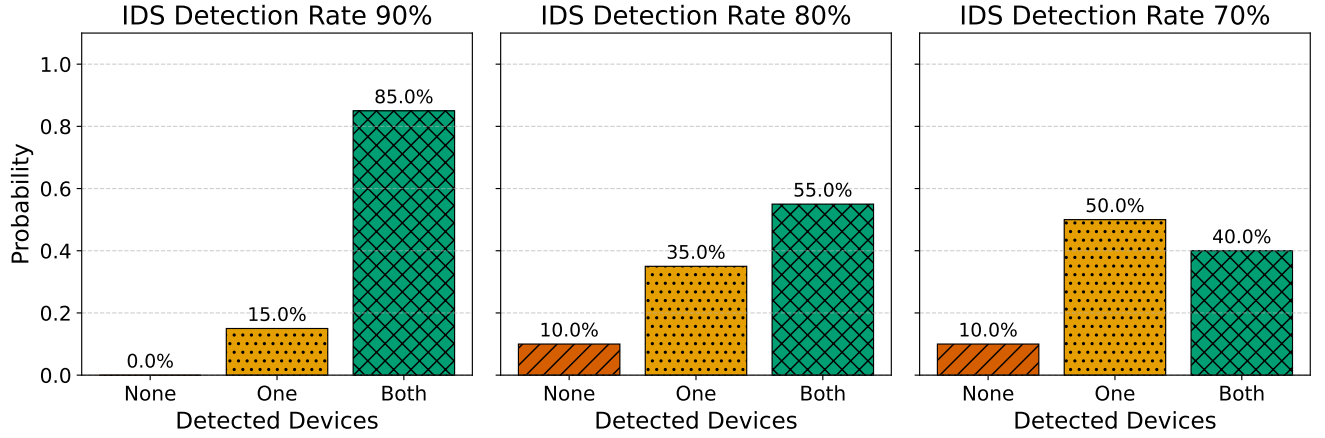
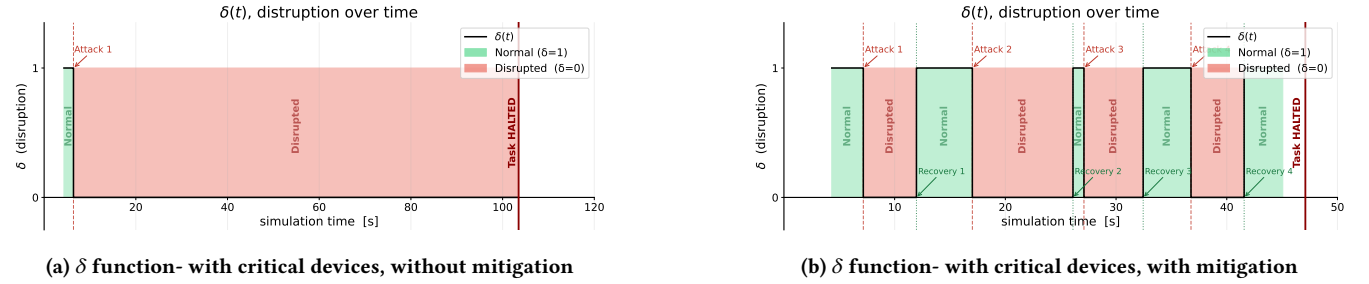
Figure 4: Timeline of the monotonic degradation function (ψ) for Experiment 9. The plot illustrates the system's performance degradation over time under various attacks: left and right wheels, left and right arms, left gripper with mitigation.

The Figure 5 share identical task and configurations but differ in mitigation scope; the plot 5a enables recovery across all actuator groups, while The Figure 5b execute experiment without any mitigation on the critical devices. As shown in both figures, this distinction has a decisive effect on system behaviour. With full coverage, four attack-recovery cycles occur within the 48.8 s baseline, yet accumulated disruption prevents task completion (HALTED at $t \approx 50$ s). Without mitigation, a single attack at $t \approx 5$ s leaves the robot permanently disrupted, no recovery event occurs. And, the task runs in a degraded state until HALTED at $t \approx 103$ s.

In Figure 6 isolate the effect of attack severity on tolerability in the absence of any mitigation (enabled_devices = $\emptyset$). Both runs share the same navigate_to_goal task, targeted devices (right_wheels, right_arm), and tolerability thresholds ($\theta_{crit} = 0.8$, $\theta_{base} = 0.72$), differing only in the base degradation rate: $\alpha_{base} = 0.04$

Table 4: Sufficient number of experiments for different Detection Rates

Detection Rate	#Run	Conf. Interval	Mean	Std	Margin Err.
80%	20	90%	0.79	0.06	0.02
80%	20	95%	0.79	0.06	0.02
80%	20	99%	0.79	0.06	0.01
70%	20	90%	0.89	0.03	0.02
70%	20	95%	0.89	0.03	0.02
70%	20	99%	0.89	0.03	0.01
Confidence Level		95%	Margin of Error ≤ 0.05		

**Figure 2: IDS detection outcomes at different κ_{crit} thresholds (0.3, 0.2, 0.1), corresponding to 70%, 80%, and 90% detection rates for two devices: Left Wheel (LW) and Right Wheel (RW).****Figure 5: Comparison of δ function- with critical devices: (a) without mitigation and (b) with mitigation.**

in Figure 6a versus $\alpha_{base} = 0.20$ in in Figure 6b. As shown in Figure 6a, the low degradation rate confines $\psi(t)$ to ≈ 0.93 throughout the 160 s run, well above θ_{crit} , placing the system in the *tolerable* region ($\gamma = 1$) regardless of attack type. In contrast, Figure 6b shows that the higher degradation rate drives $\psi(t)$ below $\theta_{base} = 0.72$ by $t \approx 9$ s, settling at ≈ 0.67 and rendering the system *not tolerable* ($\gamma = 0$) for the remainder of the run. These results confirm that without mitigation, the tolerable and intolerable boundary is governed not by which devices are attacked but by how severely performance

degrades: a mild attack on non-critical devices remains within acceptable bounds, while a more aggressive degradation on the same devices breaches the baseline threshold irreversibly.

Experiment in Figure 6a extends the comparison by reintroducing mitigation under the same aggressive degradation regime as experiment in Figure 6b ($\alpha_{base} = 0.20$, right_wheels and right_arm targeted). As shown in Figure ??, $\psi(t)$ undergoes an identical initial drop, briefly crossing below $\theta_{base} = 0.72$ to ≈ 0.67 at $t \approx 5$ s, the same trajectory that left experiment in Figure 6b permanently

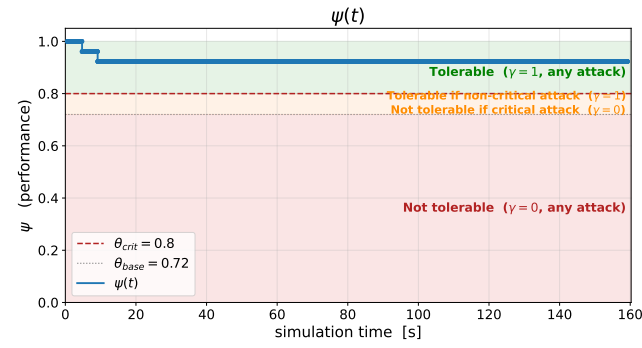
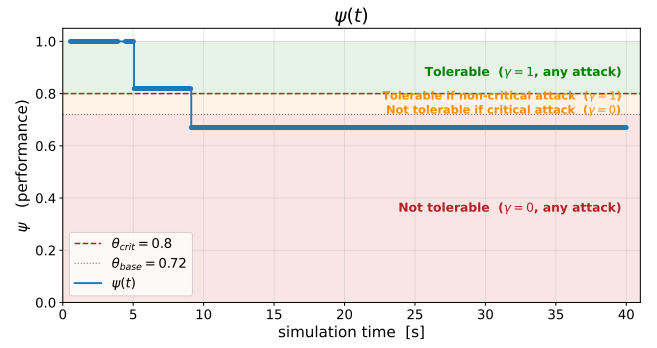
(a) ϕ function over time with non-critical devices, without mitigation(b) ϕ function over time with non-critical devices, without mitigation

Figure 6: Comparison of ϕ function over time with critical devices (multiple attacks on right wheels and right arm), without mitigation (a) tolerable (b) not tolerable

intolerable. However, mitigation applied to the attacked devices arrests the decline and recovers $\psi(t)$ to ≈ 0.81 , just above $\theta_{crit} = 0.8$, where it stabilizes for the remainder of the 220 s run ($\gamma = 1$). Together, Figure 6 form a controlled triplet: a mild attack without mitigation remains tolerable (a), a severe attack without mitigation becomes permanently intolerable (b), and the same severe attack with targeted mitigation is recovered to the tolerable zone (Figure 7), demonstrating that timely mitigation on the attacked devices is both necessary and sufficient to restore acceptable performance when degradation would otherwise breach θ_{base} .

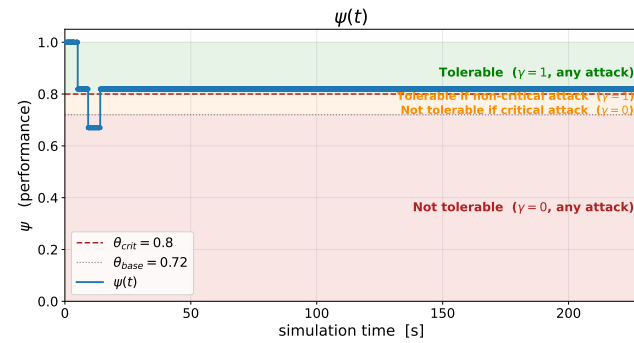


Figure 7: Timeline of the monotonic degradation function (ψ) for cyber attacking on non-critical devices. The plot illustrates the system's performance degradation over time under attacks of non-critical devices: right wheels and right arm.

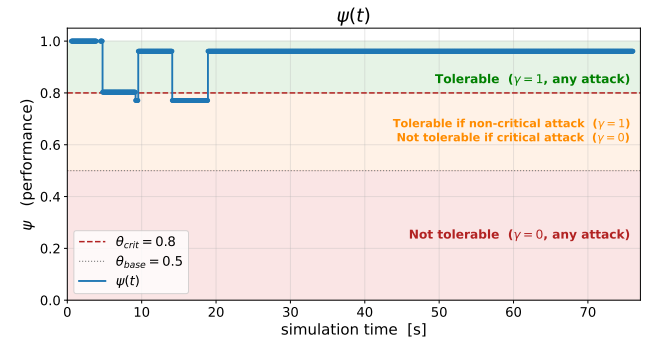


Figure 10: Timeline of the monotonic degradation function (ψ) for Experiment 18. The plot illustrates the system's performance degradation over time under various attacks on critical device left wheels, not critical device right wheel.

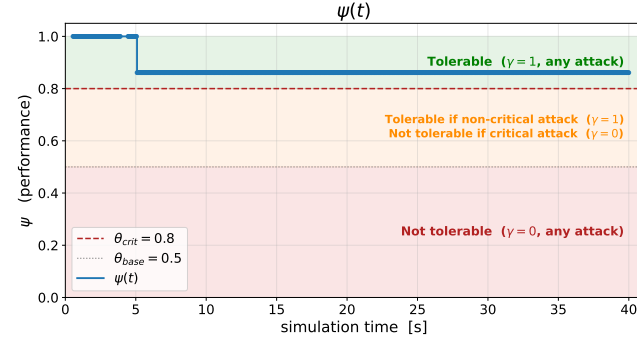
7.4 Task Sensitivity Analysis

Results, presented in 5, indicate a clear relationship between the number of critical components and the likelihood of disruption. As the number of critical components increases, the probability that a randomly selected attack combination includes at least one critical component also increases.

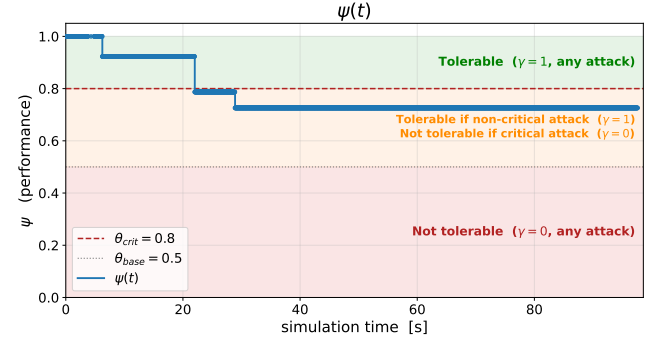
For the pickup_object task, only two components are classified as critical, resulting in a larger proportion of attack combinations that do not lead to disruption. In contrast, the navigate_and_pickup task depends simultaneously on both navigation and manipulation capabilities, increasing the number of critical components and therefore making disruption significantly more likely under arbitrary attacks. These results suggest that resilience should not be considered a static property of the robot hardware alone. Although the evaluated robot platform remains unchanged across all experiments, the disruption sensitivity varies depending on the active task configuration and its associated criticality mapping.

8 Conclusion

- summary of contributions

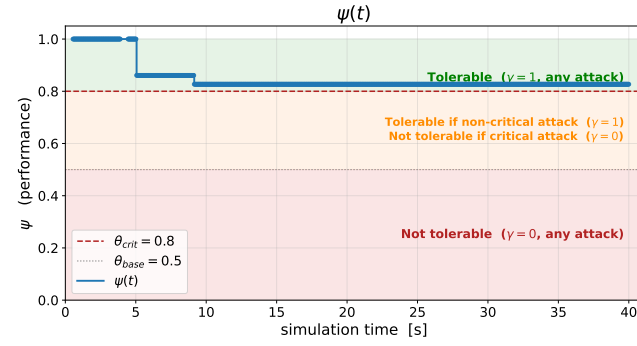


(a) ϕ function over time with critical device (left wheel), without mitigation

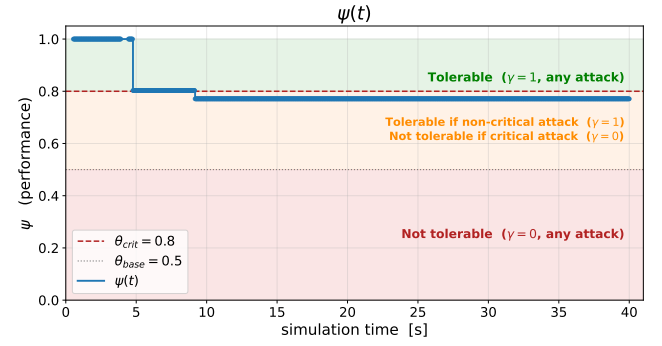


(b) ϕ function over time with critical devices (multiple attacks on left wheels, left gripper, left and right arms), without mitigation

Figure 8: Comparison of ϕ function over time with critical devices (multiple attacks on left wheels, left gripper, left and right arms), without mitigation (a) tolerable (b) not tolerable



(a) ϕ tolerable function over time with critical device (left wheel), and non-critical (right wheel) without mitigation



(b) ϕ not tolerable function over time with critical device (left wheel) and non-critical device (right wheel), without mitigation

Figure 9: Comparison of ϕ function over time with critical and non-critical devices, without mitigation (a) tolerable (b) not tolerable. To maintain in tolerable state, we set the $\alpha_{crit} = 0.15$ and in not tolerable state, we set the $\alpha_{crit} = 0.22$

Table 5: Disruption sensitivity across task types

Task Type	Critical Devices	Disrupted	Not Disrupted
pickup_object	2	96 (75.6%)	31
navigate_to_goal	3	112 (88.2%)	15
navigate_and_pickup	5	124 (97.6%)	3

- future work

References

- [1] Patricia S. Abril and Robert Plant. 2007. The patent holder's dilemma: Buy, sell, or troll? *Commun. ACM* 50, 1 (Jan. 2007), 36–44. doi:10.1145/1188913.1188915

Ethics and Privacy Statement

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna,

id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A Appendix

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Online Resources

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit

congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.